TAMAGOTCHI LIFE

ЭТАП 2 / КОМАНДНЫЙ PLAYBOOK

Инструкция по переходу от первой рабочей версии к модульной игре с Cat, Dog, GameClock и сохранениями.

Для кого: разработчики A–H, тимлид и участник финальной сборки. Основа: презентация «Занятие 05» и архив Tomagochi Life.

Что должно получиться

• Игра запускается через main.py и позволяет создать кошку или собаку.

• Cat и Dog наследуются от общего Pet, но хотя бы одно действие выполняют по-разному.

• Game управляет питомцем и отдельными часами GameClock.

• Состояние сохраняется и восстанавливается; отсутствие или повреждение файла не приводит к аварийному завершению.

• Все модули импортируются без ошибок; команда демонстрирует полный сценарий.

Маршрут команды

01 Согласовать интерфейсы → 02 Разделить файлы → 03 Реализовать роли → 04 Интегрировать → 05 Протестировать → 06 Показать результат.

Главное правило

Не переписываем всё с нуля. Переносим рабочую логику первой версии, устраняем противоречия и только затем расширяем функциональность.

01 / Что реально есть в архиве

Файл	Найдено	Что сделать
Apart.py	Pet: имя, голод, настроение, энергия, сон, час, _clamp().	Основа pets/pet.py; добавить согласованные действия и состояния.
Bpart.py	Функции feed/play/sleep/wake_up вне класса.	Перенести методы внутрь Pet; проверить отступы и исключения.
Cpart.py	Собственный Pet с временем и меню; импортирует Pet, затем переопределяет его.	Не использовать как GameClock без переработки; выделить часы и полезную логику.
Dpart.py	Исключения и валидация имени/чисел.	Перенести в exceptions.py, унифицировать классы ошибок.
Epart.py	Меню с вызовами действий; импортирует отсутствующие Cat, Dog, GameClock.	Адаптировать меню к Game и новым путям импорта.
Main.py	Создание Pet, но menu() содержит pass.	Заменить рабочей точкой входа.
Pets/dog.py	Dog с переопределёнными действиями.	Добавить импорт Pet, проверить поля и вызовы super().
requirements.txt	Есть colorama и python-dotenv.	Пересохранить в UTF-8 и проверить необходимость зависимостей.

Важно: в архиве не обнаружена законченная реализация Cat, GameClock и Storage в целевой структуре. Примеры ниже — предлагаемый контракт интеграции, а не утверждение о готовом коде.

02 / Целевая структура

```
tamagotchi/
  main.py            # запуск
  game.py            # сценарий и меню
  clock.py           # игровые часы
  storage.py         # save/load
  exceptions.py      # ошибки
  pets/
    __init__.py
    pet.py           # базовый Pet
    cat.py           # Cat(Pet)
    dog.py           # Dog(Pet)
  requirements.txt
  README.md
```

03 / Сначала согласуйте контракт

Тимлид публикует единые имена и параметры до параллельной разработки. Ниже — рекомендуемый вариант; менять его можно только согласованно.

```
Pet(name: str)
  fields: name, hunger, mood, energy, is_sleeping, is_alive
  methods: feed(), play(), sleep(), wake_up(), __str__()
Cat(Pet), Dog(Pet): override play() [or feed()]
GameClock(start_hour: int = 12)
  methods: advance(hours: float), display()
Game(pet: Pet, clock: GameClock)
  methods: run(), apply_action(action)
Storage.save(game, path="save.json")
Storage.load(path="save.json") -> Game
```

Решение по времени: в первой версии время находится внутри Pet; на втором этапе владельцем времени становится GameClock. Только Game инициирует продвижение часов после действий. Не дублировать часы в Pet.

Решение по состояниям: использовать единое имя is_sleeping (в старом коде также встречается is_sleep). Уточнить начальные значения и последствия действий до объединения.

04 / Задачи по ролям

Роль	Зона ответственности	Готово, когда…
A	pets/pet.py: базовый класс, общие поля, ограничения 0–10, действия.	Создание Pet и методы работают независимо от меню.
B	pets/cat.py: Cat(Pet), собственное поведение.	Cat наследуется и отличается от Dog.
C	pets/dog.py: Dog(Pet), перенос текущей заготовки.	Нет NameError; вызовы super() корректны.
D	clock.py: GameClock, формат и продвижение времени.	Часы не зависят от конкретного питомца.
E	storage.py: JSON save/load, восстановление типа и параметров.	Сохранение и загрузка не теряют состояние.
F	game.py и меню, выбор питомца, обработка действий.	Проходится пользовательский сценарий.
G	Импорты, структура, venv, requirements, main.py.	Проект запускается одной командой.
H	Тесты, интеграция, демонстрация, приёмка.	Весь чек-лист пройден.

05 / Инструкции по реализации

A + B + C: питомцы

• A переносит общие поля и действия в Pet, задаёт ограничения статистик и понятные исключения.

• B реализует Cat(Pet), C — Dog(Pet). Отличия поведения видны в числах или сообщениях, а не только в названии класса.

• После super().play() проверяйте, было ли действие успешным: если родитель отказал из-за сна/усталости, бонусы наследника начислять нельзя.

• Не храните время одновременно в Pet и GameClock.

D: часы

GameClock хранит игровое время; метод advance(hours) принимает только положительное число. Отображение — например, 12:00. Логику изменения потребностей при течении времени согласовать с A и F.

E: сохранение

Пример структуры save.json (контракт команды):

```
{
  "version": 2,
  "pet_type": "Dog",
  "pet": {"name": "Бим", "hunger": 5,
          "mood": 7, "energy": 6,
          "is_sleeping": false, "is_alive": true},
  "clock": {"hour": 12}
}
```

Сохраняйте только JSON-совместимые данные, а не Python-объект напрямую. При загрузке проверяйте тип, обязательные поля и диапазоны. Отсутствующий файл → новая игра; повреждённый JSON → понятное сообщение и безопасный выбор новой игры.

F + G: сборка

F создаёт Game и пользовательский цикл; G проверяет абсолютные/пакетные импорты, устанавливает зависимости и обеспечивает запуск из корня. Не оставлять два конкурирующих файла запуска.

```
python -m venv .venv
# Windows: .venv\Scripts\activate
# macOS / Linux: source .venv/bin/activate
python -m pip install -r requirements.txt
python main.py
```

Команды выше рассчитаны на запуск из папки tamagotchi. Если выбрана пакетная схема запуска, команда должна зафиксировать её в README.

06 / Как передавать работу

Код и статусы публикуем в общем RocketChat, а не в личных сообщениях. Готовые части сразу передаются интегратору Н.

Фамилия Имя — роль С
Статус: готово / нужна помощь / на проверке
Файл: pets/dog.py
Реализовано: Dog.play(), Dog.feed()
Зависимости: Pet.play(), Pet._clamp()
Проверено: создание, play(), feed()
Передано: Н
Комментарий: использует super().play()

07 / Приёмочные тесты

[ ] main.py стартует из чистого окружения.

[ ] Создаются Cat и Dog; их play() действительно различается.

[ ] Game содержит Pet и GameClock.

[ ] Кормление, игра, сон, пробуждение и пропуск времени работают.

[ ] Статистики не выходят за допустимые границы.

[ ] save → выход → load восстанавливает тип питомца, значения и время.

[ ] Без save.json игра не падает.

[ ] Повреждённый save.json обрабатывается безопасно.

[ ] Некорректный ввод не завершает приложение трассировкой.

[ ] requirements.txt устанавливается, импорты разрешаются.

08 / Сценарий финального показа

1. Показать дерево файлов → 2. Запустить игру → 3. Создать Cat → 4. Продемонстрировать действие → 5. Создать Dog и показать отличие → 6. Пропустить время → 7. Сохранить → 8. Перезапустить и загрузить → 9. Показать поведение при отсутствующем файле.

После демонстрации тимлид фиксирует

Что работает / что осталось доделать / кто отвечает / до какого срока. Дополнительные задачи (colorama, третий питомец, сложность, расширенные тесты, README) — только после обязательного функционала.

Источники: «Занятие 05. Проект Тамагочи этап 2 — практика», стр. 2–12; архив Tomagochi Life, файлы assets/scripts/*.py, Pets/dog.py и requirements.txt.